

Skullbonez Core

Technical Manual

Engine architecture, physics, rendering, diagnostics, and improvement notes

Whole-Engine Ownership Map

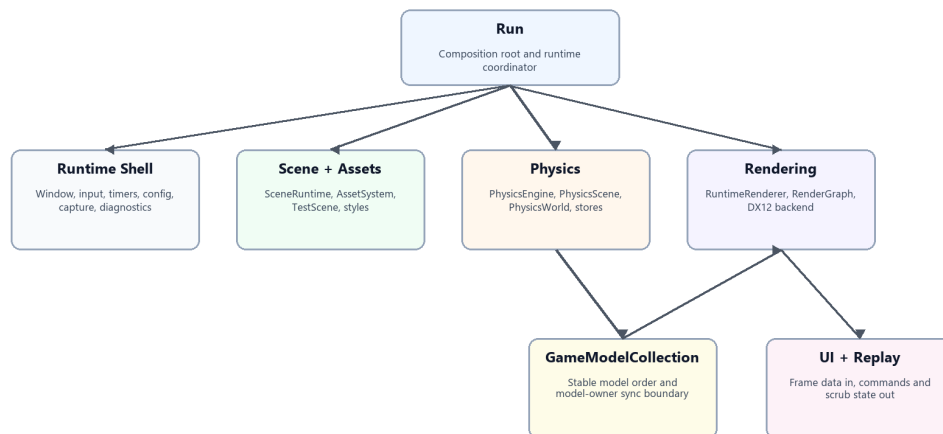


Figure 0.1 - The manual's top-level map.

Prepared July 2026 from the current SkullbonezCore repository state.

Contents

Print recommendation

Use the DOCX as the editable master and this PDF as the Officeworks print copy. The PDF fixes pagination, figures, and equation cards for predictable output.

1. Chapter 1: Engine Architecture - Skullbonez Core is easiest to understand as a runtime composition root wrapped around explicit subsystem boundaries. Run still coordinates the frame, but newer facades name the work that is being carved out of it.
2. Chapter 2: Scene, Assets, and Runtime State - Scenes are declarative inputs. They become runtime state only through parser, asset, setup, and model-owner boundaries.
3. Chapter 3: Physics - Physics is a fixed-step, deterministic data pipeline. Forces update velocities first; contact rows then solve impulses directly into velocities; final pose integration uses those solved velocities.
4. Chapter 4: Rendering - Rendering is a DX12-first pass pipeline wrapped by RuntimeRenderer. The render graph is the vocabulary for resource access; the DX12 backend turns that vocabulary into explicit barriers, descriptors, uploads, fences, and presents.
5. Chapter 5: Diagnostics, Replay, and Validation - Skullbonez treats evidence as part of the engine. Diagnostics, replay, and validation gates exist so changes can be understood, reproduced, and bounded.
6. Chapter 6: Working With the Engine - The best way to change Skullbonez Core is to respect ownership, validate the specific risk, and leave better explanations than you found.

How to Read This Manual

- Start each chapter with its mental model, then use diagrams to place the subsystem in the engine.
- Read equations as implementation anchors, not as a claim that every line mirrors a textbook derivation.
- Use each Room for Improvement section as a prompt list for future plans or PR review questions.
- When source and manual disagree, trust the current source and revise the manual.

Chapter 1: Engine Architecture

Mental model

Skullbonez Core is easiest to understand as a runtime composition root wrapped around explicit subsystem boundaries. Run still coordinates the frame, but newer facades name the work that is being carved out of it.

Whole-Engine Ownership Map

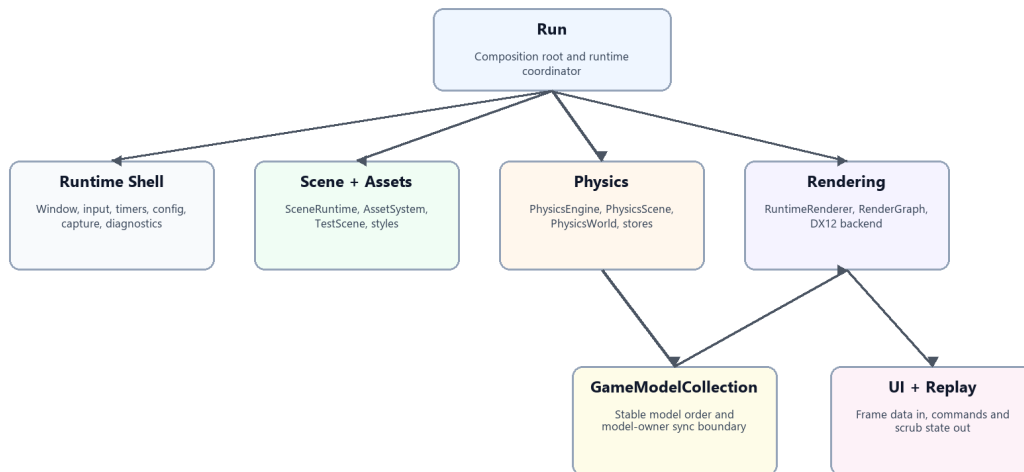


Figure 1.1 - Engine Architecture overview.

The Composition Root

Run owns the long-lived runtime graph: native window access, config, scene queue, cameras, world environment, game models, UI, replay, capture, diagnostics, and the renderer host. It is still broad, but the code now labels its borrowed boundaries so future extraction work has real names.

Reader note

Reading rule: before changing a subsystem, identify the owner, the borrowed view, and the writeback point.

The key habit is to read ownership before reading behavior. EngineContext is a bound view over Run-owned systems. RuntimeRenderInputs is rebuilt for one render call and discarded. PhysicsModelAccess is the model-owner sync boundary between legacy GameModel storage and physics stores.

That pattern lets the engine modernize without pretending the old global shape vanished overnight. A facade is useful only when it tells the truth about ownership and lifetime.

- Run is the composition root, not a domain model.
- SceneRuntime owns active scene and queue state, while Run still applies heavy load/reset side effects.
- SimulationSystem owns timestep policy and accumulators.
- RuntimeRenderer owns pass objects and the ordered render frame.

The Frame as a Contract

A frame starts with input and runtime commands, not with rendering. InputController captures key and mouse edges; UI emits command intent; scene and replay code may change the active state before simulation asks for time.

SimulationSystem then converts wall time or fixed-step mode into zero or more physics ticks. Camera and UI use a frame-level delta, but physics advances in fixed units so solver behavior stays reproducible.

Rendering follows after simulation and replay preview state have been applied. Run sets the active camera, builds RuntimeRenderInputs, calls RuntimeRenderer::RenderFrame, then restores any replay pose that was temporarily borrowed for presentation.

Frame Loop and Fixed-Tick Dispatch

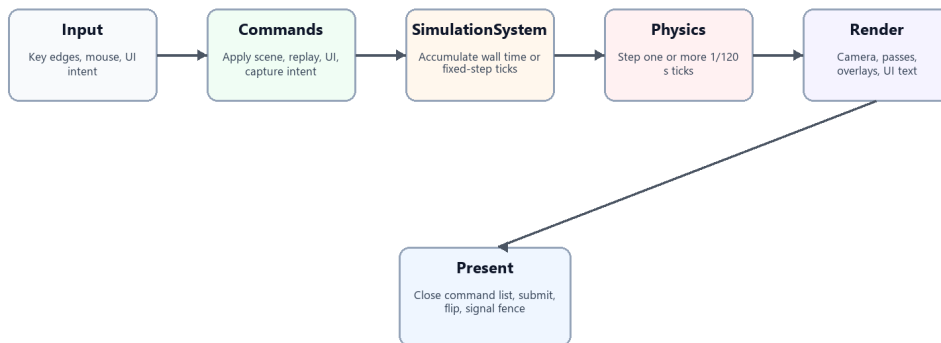


Figure - The Frame as a Contract.

Fixed timestep accumulator

```

acc = acc + frame_dt * time_scale
while acc >= h: Step(h); acc = acc - h
Skullbonez physics tick: h = 1 / 120 seconds
  
```

Equation - The Frame as a Contract.

Stable Indices and Data Boundaries

GameModelCollection is the stable owner of model order. Physics stores, render streams, debug overlays, replay samples, and scene snapshots all depend on that deterministic ordering.

The current engine still mirrors solved physics back into GameModel because editor, replay, render, and diagnostics consumers read model state. The important architectural progress is that the solver itself runs over PhysicsBodyStore, ColliderStore, compact streams, and bounded side-effect arrays.

This is why migration cleanup focuses so much on authority. A hot loop should not reach through Run or GameModel; it should consume compact value records and let the owner apply side effects after the pass.

- Authoritative legacy storage: GameModel vector inside GameModelCollection.
- Physics hot path: body records, collider records, body streams, solver scratch, side-effect buffers.
- Render path: render instance streams and per-frame RuntimeRenderInputs.
- Diagnostics path: bounded pipeline records and queryable SkullScope traces.

Subsystem Extraction Philosophy

The codebase is not doing a big-bang rewrite. Instead, it names a boundary, preserves behavior, proves the validation surface, and then tightens authority in later slices.

Good extraction removes hidden reach-through. Bad extraction merely gives the old reach-through a new name. The repository rules deliberately discourage vague compatibility nouns because a bridge without a deletion condition can become permanent architecture.

This matters most in physics and rendering. Both are hot-path systems where callback chains, polymorphic services, and owner-side commands can silently turn a deterministic loop into scattered state mutation.

Pattern	Useful When	Risk
Facade	It names a real boundary and forwards without reordering.	It can hide old ownership if the next owner is not clear.
Value record	Hot path needs predictable memory and deterministic iteration.	It must be refreshed and invalidated at the right times.
Side-effect queue	Solver or worker pass needs to report owner mutations later.	It needs bounded capacity and deterministic commit order.

The Mental Stack

At the top is runtime intent: command-line flags, scene JSON, UI commands, replay state, and validation harnesses. In the middle are owner subsystems that translate intent into current state. At the bottom are hot loops that should see only the data they need.

Most engine questions become easier when you place them on that stack. A scene option belongs near parsing and setup. A DX12 resource state belongs in graph/backend code. A contact impulse belongs in solver scratch until the writeback stage.

The manual follows that stack throughout: ownership first, data movement second, equations third, improvement notes last.

- Intent: command-line, scene JSON, UI, replay, validation.
- Owners: Run, SceneRuntime, SimulationSystem, PhysicsScene, RuntimeRenderer, AssetSystem.
- Hot data: body records, collider records, render instances, descriptor rows, upload arenas.

Room for Improvement

The list below is deliberately framed as engineering opportunity rather than criticism. It names the places where the chapter's current architecture can become clearer, smaller, or easier to validate.

	Improvement prompt
[]	Continue shrinking Run by moving load/reset side effects behind named scene/runtime coordinators.
[]	Finish strict authority work so physics and rendering consume stores/streams without legacy model reach-through.
[]	Keep facades honest with owner, reason, deletion condition, and checker budget when they are transitional.
[]	Add more human-facing diagrams beside the class-structure reference as subsystems stabilize.

Chapter 2: Scene, Assets, and Runtime State

Mental model

Scenes are declarative inputs. They become runtime state only through parser, asset, setup, and model-owner boundaries.

Scene and Asset Data Flow

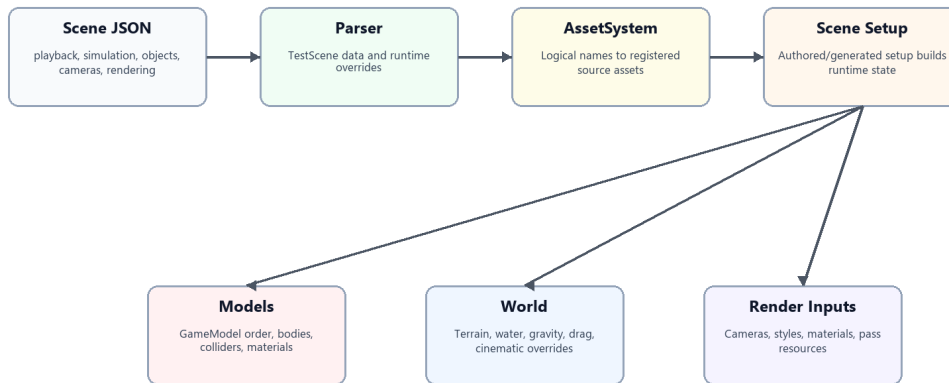


Figure 2.1 - Scene, Assets, and Runtime State overview.

Scene Files as Runtime Requests

A Skullbonez scene file is a request, not a live object graph. It can describe playback, simulation, world overrides, cameras, objects, terrain, rendering, UI state, and validation gates.

SceneRuntime and SceneController hold the active queue/index state, while authored and generated setup helpers build the runtime objects. This keeps scene selection separate from the actual mutation that creates terrain, models, cameras, and validation records.

The runtime also accepts suites, generated demo modes, hero scene aliases, replay probes, live style control, and validation-only launch flags. All of those paths eventually have to converge on the same state ownership rules.

- Scene JSON is versioned declarative data.
- Suites sequence scene files for validation and render capture.
- Style files layer cinematic/material settings without rebuilding physics.
- Snapshot writing serializes current state back to scene JSON when requested.

Asset Resolution

AssetSystem owns source asset identities and resolves logical names to data-root paths. Textures, shader programs, style files, scenes, asset libraries, terrain, and fonts all become source records before runtime code asks the backend to create GPU objects.

The asset registry is intentionally separate from backend memory. A texture source asset says what to load and how; the DX12 texture entry, descriptors, and resource lifetime are renderer concerns.

Reusable placeables should be registered asset libraries. Scenes should prefer assetInstances for reusable props, hulls, and compound structures instead of baking every part into scene object lists.

Asset Kind	Runtime Use	Owner Boundary
Texture2D	Materials, sky, UI, water, terrain.	AssetSystem source record; renderer GPU lifetime.
ShaderProgram	Logical shader names and contracts.	AssetSystem resolves; ShaderDX12 compiles and reflects.
AssetLibrary	Reusable convex hulls and compounds.	Registered from Run setup, expanded by scene setup.

GameModelCollection as Traffic Controller

GameModelCollection is where authored objects become a deterministic order that physics, rendering, replay, and diagnostics can agree on. It is not merely a container; it is the model-owner boundary.

PhysicsModelAccess exposes the operations physics is allowed to perform: reload bodies, refresh colliders, write back one body or all bodies, access streams, and invalidate stream caches after solver mutation.

Render code does not need the entire collection as an owner; it needs per-frame views and render instances. That is why GameModelStreamProvider and RuntimeRenderInputs matter: they mark the path from historical objects toward view-shaped data.

- Preserve model insertion order for deterministic solver and replay behavior.
- Refresh collider snapshots when topology changes, not inside every row solve.
- Invalidate derived streams after physics writeback.
- Keep editor/replay mirrors outside the narrow solver loop.

World State

WorldEnvironment names gravity, fluid, drag, water height, and related runtime forces. Physics receives a world-force snapshot so the solver does not reach back into world ownership during the hot pass.

Reader note

Good boundary test: a render-only scene style should not reset physics, rebuild bodies, or change the physics regression CSV.

Terrain has two jobs: render geometry and collision surface. The physics path treats terrain contact as a borrowed per-call view and produces TerrainContactManifold value reports rather than mutating a full scene object.

Cinematic overrides change render presentation, not physics terrain. This separation is important: visual relief, fog, bloom, sky atmosphere, and water presentation must not secretly alter deterministic physics baselines.

Velocity-first integration

$$v_next = v + h * M_inverse * F$$

$$x_next = x + h * v_next$$

orientation advances from angular velocity after solver velocity writeback

Equation - World State.

Runtime Control Surfaces

Skullbonez has several control surfaces: command-line flags, scene directives, UI tabs, replay scrub controls, style harness files, and validation scripts. Their common task is to express intent without breaking subsystem ownership.

For example, the live style harness applies style-only JSON without reloading physics. Replay scrub temporarily applies presentation or solver samples for rendering, then restores live state after the frame. Validation launches use explicit flags so artifacts can be reproduced.

The more control surfaces a runtime has, the more valuable small owner-shaped structs become. They make each mutation explainable and reduce the chance that a convenience path bypasses the normal gate.

- Command-line flags are public compatibility surface.
- UI emits commands; Run/runtime controllers apply them.
- Replay preview borrows render state and restores it.
- Validation scenes encode evidence needs in data.

Room for Improvement

The list below is deliberately framed as engineering opportunity rather than criticism. It names the places where the chapter's current architecture can become clearer, smaller, or easier to validate.

	Improvement prompt
[]	Move more scene reset/load side effects out of Run into scene-owned coordinators.
[]	Keep asset library registration data-driven and avoid new editor-only hardcoded compound objects.
[]	Broaden source diagrams for scene parsing, style layering, and snapshot writing.
[]	Add targeted docs for assetInstance authoring, hull baking, and reusable prop validation.

Chapter 3: Physics

Mental model

Physics is a fixed-step, deterministic data pipeline. Forces update velocities first; contact rows then solve impulses directly into velocities; final pose integration uses those solved velocities.

Fixed Physics Tick Pipeline

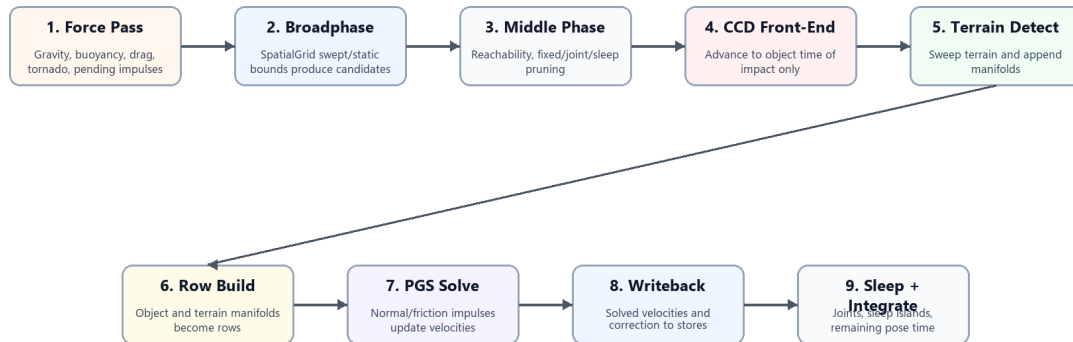


Figure 3.1 - Physics overview.

Fixed Tick Dispatch

SimulationSystem owns the policy that turns frame time into physics steps. In deterministic fixed-step mode it ignores wall-clock accumulation and commits whole PHYSICS_FIXED_DT ticks from the time-scale accumulator. In variable-time scenes it still runs physics in fixed-size steps, capped to avoid runaway catch-up.

The important split is that physics receives a fixed dt while camera/UI presentation can use a frame-level dt. This is why scene validation can reproduce byte-exact physics behavior even when rendering cadence changes.

Every fixed tick runs through GameModelCollection::RunPhysics, PhysicsEngine::Step, PhysicsScene::RunPhysics, and PhysicsWorld::RunPhysics. PhysicsScene reloads body records, refreshes colliders when topology changed, runs the world, copies sleep state, and writes body records back to models once.

Fixed timestep accumulator

```
acc = acc + frame_dt * time_scale
while acc >= h: Step(h); acc = acc - h
Skullbonez physics tick: h = 1 / 120 seconds
```

Equation - Fixed Tick Dispatch.

Layer	Responsibility
SimulationSystem	Accumulates time and decides how many fixed ticks to commit.
PhysicsEngine	Runtime-facing facade over PhysicsScene.
PhysicsScene	Owns body/collider stores and model-owner synchronization.
PhysicsWorld	Runs broadphase, narrowphase, solver, sleep, diagnostics.

Force Pass

The first major physics phase applies world forces to awake dynamic bodies. Fixed bodies return immediately. Sleeping bodies keep cached state and have their remaining time cleared until a contact or scene change wakes them.

Forces include gravity, buoyancy, drag, tornado effects, and pending impulses. These forces update linear and angular velocity. They do not resolve contacts.

This ordering matters because contact rows solve against the velocities that already include world forces. Gravity makes a body want to fall; the contact solver then produces impulses that oppose that motion when a support row exists.

Velocity-first integration

$$v_{\text{next}} = v + h * M_{\text{inverse}} * F$$

$$x_{\text{next}} = x + h * v_{\text{next}}$$

orientation advances from angular velocity after solver velocity writeback

Equation - Force Pass.

- Sleeping bodies are static anchors until wake policy admits them.
- A newly woken body applies forces immediately so it behaves like it was awake for the tick.
- Tornado and world-force effects run before broadphase and row solving.

Broadphase and Middle Phase

SpatialGrid is the cheap candidate generator. It inserts swept bounds for fast moving dynamic bodies and static bounds for ordinary bodies, then emits unique pairs from occupied cells.

Candidate pairs are not contacts. They are only pairs worth testing more precisely. The middle phase prunes pairs that cannot affect simulation: swept bounding-radius reachability rejects, fixed-fixed pairs, point-joint pairs, and sleep-sleep pairs.

The grid is still populated with sleeping bodies because an awake body must be able to find and wake a sleeping neighbor. What gets removed is work that cannot produce wake, response, or diagnostics.

- Cell size follows the largest active broadphase primitive, clamped by config.
- Fast-small sweep augmentation protects tiny high-speed projectiles.
- Pair ordering is deterministic: pairs are normalized so $A < B$.
- The middle phase may keep false positives, but it must not reject real possible contacts.

Object CCD Front-End

The object/object swept front-end is continuous collision detection for timing and wake-up. It can refine a time of impact and advance bodies to the contact candidate, but it does not apply object/object collision response.

Reader note

Short version: CCD finds when to meet; persistent rows decide how the bodies respond.

That distinction is the core answer to the question that inspired this manual: contact rows do not become forces that later pass through $F = ma$. The world force pass already happened. Contact rows become impulses applied directly to solver velocities.

If a pair is already persistent and motion is small, CCD may skip the swept front-end and let the persistent manifold handle the settled contact. New or fast contacts keep the swept path so tunneling is avoided.

Anatomy of a Persistent Contact Row

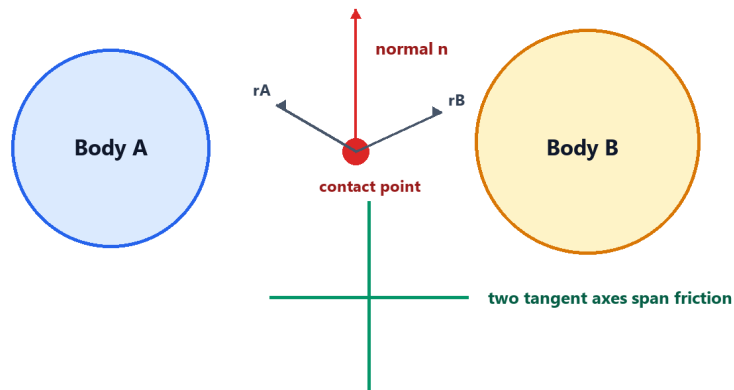


Figure - Object CCD Front-End.

Contact point velocity

$$v_{A_p} = v_A + \omega_A \text{ cross } r_A$$

$$v_{B_p} = v_B + \omega_B \text{ cross } r_B$$

$$v_{\text{rel}} = v_{B_p} - v_{A_p}, \quad v_n = \text{dot}(v_{\text{rel}}, n)$$

Equation - Object CCD Front-End.

Terrain Detection and Manifolds

Terrain is still detected with a swept path before the shared solve, because fast bodies need a correct time of impact. But the terrain phase now emits terrain manifolds rather than running a separate terrain impulse response.

A `TerrainContactManifold` records `bodyA`, `bodyB = -1` for terrain, normal, tangents, contact points, time of impact, support policy, friction permission, sleep inhibition, and feature ids.

Stable resting terrain support may seed sleep and warm starting. Edge or point contacts can inhibit sleep and avoid rest-only policy while still producing rows for normal and friction response.

- Terrain body index -1 means infinite mass, zero velocity, and no writeback.
- Terrain normals are flipped when converted to shared `PersistentContact` row convention.
- Terrain warm-start seed is strongest for stable support and smaller for sleep-inhibiting edge contacts.

Manifold to Row Build

PersistentContactSolver builds object/object rows from exact shape-pair manifolds and appends terrain rows from terrain manifolds. A row carries body indices, contact normal, tangent axes, rA/rB arms, penetration, feature id, support flags, friction policy, cached impulses, and per-row masses/biases.

Object rows use exact sphere, box, and convex-hull manifold builders. Quiet same-shape face footprints can reduce many manifold points to two well-spread cached rows, which cuts solver work without losing the support plane.

Each row gets a stable key. For object/object contacts the key combines the ordered pair and feature id. For terrain rows it uses a terrain kind bit, body A, and the terrain feature id.

Persistent Contact Solver Loop

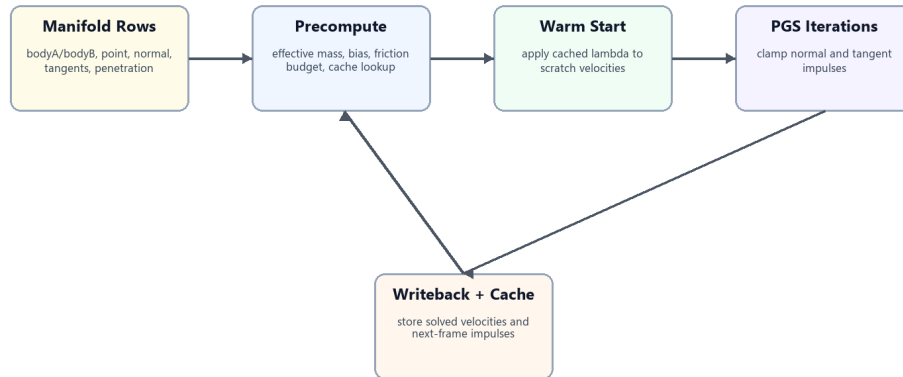


Figure - Manifold to Row Build.

Normal row impulse update

$$m_{\text{eff}} = 1 / (\text{invMassA} + \text{invMassB} + \text{angular terms})$$

$$\text{delta_lambda_n} = m_{\text{eff}} * (\text{bias} - v_n)$$

$$\text{lambda_n} = \max(0, \text{lambda_n} + \text{delta_lambda_n})$$

Equation - Manifold to Row Build.

Projected Gauss-Seidel

The row solver is Catto-style sequential impulses. Precompute builds tangent axes, normal/tangent effective masses, restitution or Baumgarte bias, friction limits, and cached warm-start impulses.

Warm starting applies cached lambda to scratch solver velocities before iteration. Then each Projected Gauss-Seidel pass recomputes contact relative velocity, solves a normal impulse, clamps accumulated normal lambda to be non-negative, solves tangent friction, clamps the two tangents as one 2D friction vector, and applies only the delta impulse.

The solver writes velocity changes to compact SolverBodyState scratch first. Only after the row loop are solved linear and angular velocities written back to PhysicsBodyRecord storage.

Tangent friction clamp

```
lambda_t = (lambda_t1, lambda_t2)
length(lambda_t) <= friction_limit
Skullbonez clamps the tangent pair as one 2D cone budget.
```

Equation - Projected Gauss-Seidel.

- Normal impulses can push apart but cannot glue together.
- Friction is bounded by row policy: terrain can use normal impulse or warm-start support; object rows may use a gravity-sized budget or normal-coupled friction.
- A deterministic early-out can stop iterations when total squared impulse delta is tiny.

Writeback, Integration, and Sleep

After solving, terrain rest policy can damp tiny supported rolling motion. Then solved velocities are written back to body records and queued for model-owner mirror writeback. Partial positional correction removes visible leftover overlap without replacing the velocity solver.

PhysicsWorld then wakes point-joint-connected sleepers, solves point joints, appends joint support edges, propagates sleep support, and integrates remaining time for awake models using the solved velocities.

Sleep is island-level. The engine builds contact and joint islands, checks quiet linear/angular thresholds, requires credible support anchors, honors terrain inhibition and point-joint error gates, then sleeps the whole eligible island only after every awake member accumulates the required quiet frames.

- Solved velocities are cached for next-frame warm starting.
- Remaining pose integration uses each body's remaining time after CCD/terrain advancement.
- Sleeping transition zeroes residual velocities to prevent tiny drift from reappearing on wake.

Room for Improvement

The list below is deliberately framed as engineering opportunity rather than criticism. It names the places where the chapter's current architecture can become clearer, smaller, or easier to validate.

	Improvement prompt
[]	Finish strict store authority so solver-facing code no longer needs legacy GameModel mirror assumptions.
[]	Continue replacing hot-path callbacks and compatibility commands with compact side-effect arrays.
[]	Consider broader awake/sleeping broadphase partitioning only with physics and perf validation evidence.
[]	Expand SkullScope canned questions for contact rows, sleep islands, and terrain support classification.
[]	Add diagrams directly beside the in-game physics pipeline overlay modes.

Chapter 4: Rendering

Mental model

Rendering is a DX12-first pass pipeline wrapped by RuntimeRenderer. The render graph is the vocabulary for resource access; the DX12 backend turns that vocabulary into explicit barriers, descriptors, uploads, fences, and presents.

Runtime Rendering Pass Order

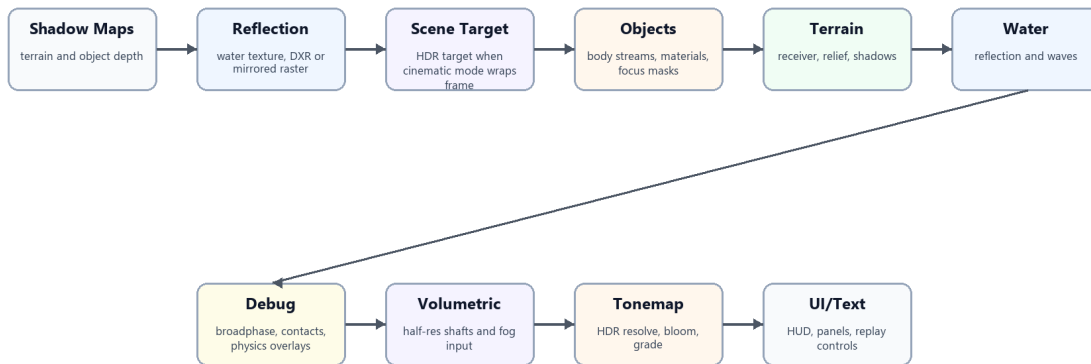


Figure 4.1 - Rendering overview.

RuntimeRenderer and Pass Ownership

RuntimeRenderer owns pass instances and the stable pass order: full-screen helpers, sky, scene target, shadows, reflection, objects, terrain, water, tornado visuals, debug overlays, volumetric light, tonemap, and UI text.

Run prepares the active camera and replay render state, checks that the process-bound backend is ready, narrows the backend into command/resource/diagnostic/raytracing facets, and calls `RuntimeRenderer::RenderFrame` with per-frame borrowed inputs.

Passes should not store broad runtime state. `RenderFrameContext` and `RenderResourceContext` separate per-frame drawing data from create/rebuild-only resource access.

World to clip space

```
p_world = M_model * p_local
```

```
p_clip = Projection * View * p_world
```

Shadow/reflection passes use alternate View matrices, not alternate objects.

Equation - RuntimeRenderer and Pass Ownership.

- `RenderFrameContext` carries matrices, light vector, water height, scene view, backend facets, and dimensions.
- `RenderResourceContext` is for resize-sensitive resources and lazy creation.
- UI text may use the optional DXR facet when the active backend publishes it.

DX12 as the Production Backend

DX12 is the only runtime renderer. The historical GL and DX11 parity evidence is archived; renderer regression evidence now means DX12 validation errors plus DX12 screenshot/baseline comparisons.

RenderBackendDX12 implements the engine-facing renderer facade and owns explicit DX12 machinery: device/queue/swapchain/fence timeline, descriptor allocators, upload arenas, readback buffers, command list state, pipeline state cache, textures, meshes, framebuffers, and optional DXR structures.

The backend is deliberately explicit because DX12 makes resource state, descriptor lifetime, upload memory, and CPU/GPU synchronization visible. Hidden state is where GPU races like to hide.

DX12 Piece	What It Owns
Dx12RenderDevice	Factory/device/queue/swap chain/fence lifetime.
Descriptor allocators	SRV/UAV shader-visible rows plus CPU RTV/DSV rows.
Frame upload system	Per-frame upload arena and allocator pacing.
Readback buffers	Backbuffer capture and GPU timer readback.

Render Graph Access

RenderGraph is the API-neutral contract for resources, passes, uses, transitions, and transient lifetimes. The current compiler is deliberately simple: start at each resource initial access, walk passes in added order, process reads and writes, emit transitions when desired access differs, and remember the new access.

The graph does not execute arbitrary runtime behavior by itself. Callback-owned passes hold runtime-specific state; the graph passes them a small context with graph vocabulary and pass identity.

For DX12, graph transition records become concrete resource barriers. If a graph-owned transition does not emit exactly one concrete barrier when it should, the backend throws rather than silently continuing.

Render Graph Resource State Tracking

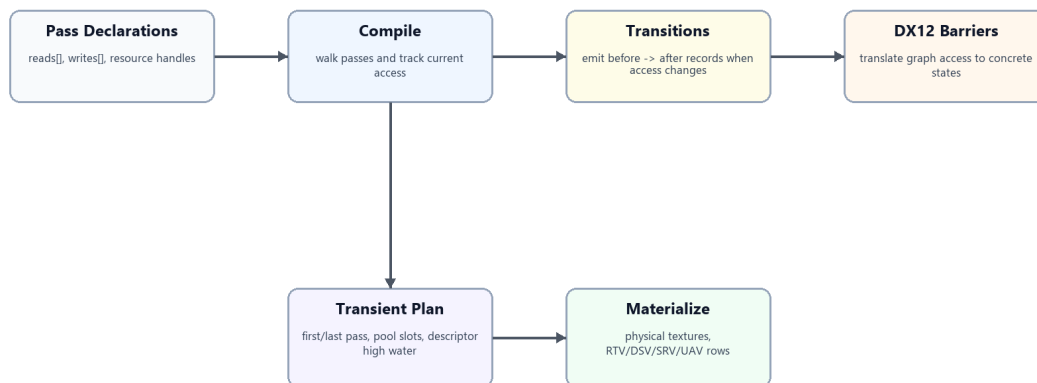


Figure - Render Graph Access.

Render graph transition rule

```

before = tracked_access(resource)
if before != requested_access: emit transition(before -> requested)
tracked_access(resource) = requested_access
  
```

Equation - Render Graph Access.

Descriptors, Uploads, and Fences

DX12 descriptor rows are binding records, not textures. Persistent texture descriptors are copied into transient shader-visible rows before draw so the command list sees a valid table for the current frame.

Constant buffers are uploaded through aligned upload memory. ShaderDX12 reflects compiled HLSL constant buffers so SetFloat, SetVec, and SetMat writes land at the offsets the shader actually compiled with.

Present closes the command list, submits it, flips the swapchain, signals a fence, advances allocator/backbuffer indices, waits only when needed before reusing allocator/upload/descriptor memory, and retires deferred releases after completion.

DX12 Frame Ownership and Lifetime

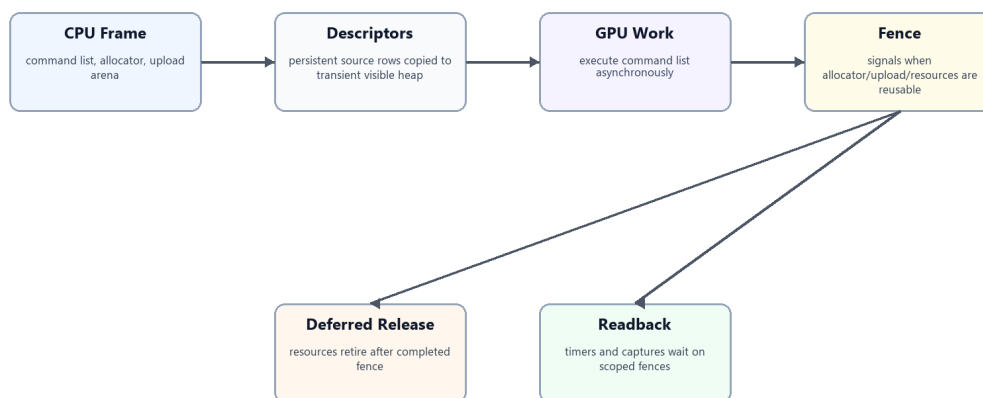


Figure - Descriptors, Uploads, and Fences.

Fence-scoped reuse

```

submit(command_list); signal(frame_fence)
reuse allocator/upload/descriptors only after completed_fence >= frame_fence
deferred releases retire on completed fences.
  
```

Equation - Descriptors, Uploads, and Fences.

Shader Contracts

Shader source assets resolve logical names. ShaderDX12 compiles vertex and pixel entry points from the HLSL file, hashes bytecode, reflects uniforms/resources, and checks debug contracts when available.

ShaderContracts.h documents the current binding ABI: ordinary raster resources use SRV register slots, with slots t0 through t4 currently exposed. t4 is the object material table while per-instance streams carry draw-local material payload.

Contracts matter because render bugs often come from mismatched assumptions: a shader expects a uniform or texture that the pass never sets, or a pass binds a texture into the wrong slot.

- Reflection discovers constant-buffer variable offsets and sizes.
- Constant buffer size is aligned to 256 bytes for DX12.
- Debug builds report missing required uniforms and reflection mismatches.

Cinematic and Debug Rendering

Cinematic rendering wraps the frame with HDR targets, sky atmosphere, clouds, god rays, volumetric light, bloom, fog, tone mapping, shadows, and terrain relief settings. These are presentation controls layered over scene state.

Debug rendering is intentionally part of the render pass order. Collision state colors, broadphase cells, contact manifolds, sleep islands, terrain probes, transparent bodies, replay paths, and UI panels all need to draw in predictable layers.

The manual distinction is simple: cinematic controls change how the world is seen; physics debug controls show how the world was simulated. Mixing those responsibilities would undermine both visual polish and deterministic evidence.

- Shadow maps work in normal and cinematic rendering.
- Reflection may use DXR or mirrored raster depending on capability and debug alpha needs.
- The tonemap pass is where HDR scene color, fog, bloom, grade, and volumetric light meet.

Room for Improvement

The list below is deliberately framed as engineering opportunity rather than criticism. It names the places where the chapter's current architecture can become clearer, smaller, or easier to validate.

	Improvement prompt
[]	Keep moving pass resource ownership from Run-shaped aggregates into renderer/pass-owned boundaries.
[]	Tighten render graph callback coverage until more passes can be fully graph-owned.
[]	Add diagrammed resource-state examples for shadow, reflection, water, and tonemap resources.
[]	Continue DX12 validation gate improvements around InfoQueue errors and screenshot timing.
[]	Preserve future portability vocabulary without adding new runtime dependency on retired backends.

Chapter 5: Diagnostics, Replay, and Validation

Mental model

Skullbonez treats evidence as part of the engine. Diagnostics, replay, and validation gates exist so changes can be understood, reproduced, and bounded.

Validation and Evidence Flow

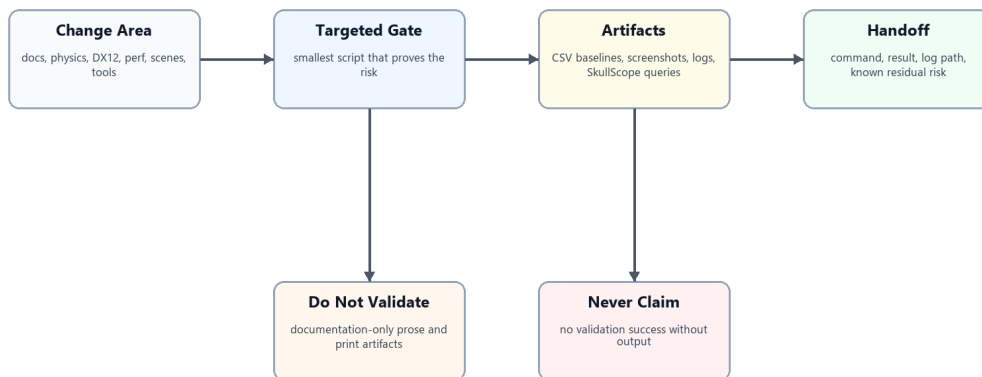


Figure 5.1 - Diagnostics, Replay, and Validation overview.

Validation Philosophy

Repository validation scripts are pre-commit and PR gates, not a reflex after every edit. The right command is the smallest command that proves the risk introduced by the change.

Documentation-only changes require no repository validation. Physics behavior changes require byte-exact physics gates. Renderer/shader/screenshot changes require DX12 renderer validation. Hot-path work may require perf validation.

The non-negotiable part is evidence: never claim validation success without command output. If validation is skipped because the work is docs-only, say that plainly.

Change Area	Evidence
Docs/manual prose	Render and visual QA for the document; no repo validation.
Physics/contact/solver	tools\validate_physics.bat and byte-exact CSV match.
DX12/shader/rendering	tools\validate_dx12_renderer.bat and zero DX12 validation errors.
Hot path/perf	tools\validate_perf.bat plus manual hot-path review.

SkullScope and Physics Queries

Physics diagnostics should stay cheap for model analysis. Instead of loading raw CSV, NDJSON, or SQLite artifacts, use SkullScope traces and query them with `tools\physics_query.bat`.

The physics pipeline overlay records bounded `pipeline_stages` rows. A query can summarize frames, events, contacts, sleep decisions, or replay restore outcomes without dumping an entire trace into a conversation.

When SkullScope is used, the handoff must report trace command, query commands, output sizes, and what data the model actually ingested. That discipline keeps diagnostics powerful without turning them into unbounded context sludge.

- Use `--physics-diag` in Debug builds to generate queryable traces.
- Prefer `summary/events/questions` queries over raw artifacts.
- Rerun narrower queries if shell output is truncated.

Replay as a Runtime Microscope

Replay capture stores recent presentation and solver samples. The presentation row previews camera/body state. The solver row stores enough authoritative physics state to restore a retained fixed tick.

Scrubbing is intentionally presentation-safe: render preview applies a historical pose, draws it, and restores live state afterward. Branching from a solver target is stricter because it mutates live simulation and must restore/check solver state.

Prediction is a sandboxed lookahead. It advances speculative physics in amortized slices, restores the real live world before rendering, and exposes profiler markers so expensive prediction work can be measured.

- Replay paths show causal body chains for selected targets.
- Velocity edit pauses simulation and redraws future paths from edited live velocity.
- V2 replay artifacts include event/checkpoint data for file-backed scrub and restore probes.

Profiler and Memory Evidence

The engine exposes CPU profiler scopes and optional platform profiler markers. Physics and rendering code names important stages so investigations can compare time against the architecture.

Graphics stress is opt-in. It churns scene loads, cinematic settings, sky/fog/ray controls, render toggles, debug overlays, object counts, UI state, and memory records from a deterministic seed.

Memory evidence separates process memory from engine buckets, DXGI usage, descriptors, upload arena pressure, and cache/pool counts. The goal is not just to know memory grew; it is to know which owner grew.

- Use platform profiler markers when changing marker code.
- Use bounded graphics stress for resource lifetime or memory-growth investigations.
- Report log paths and key lines instead of pasting giant artifacts.

Documentation as Evidence

This manual is itself a documentation artifact, so its validation is layout validation: render the DOCX, inspect pages, and provide the PDF for printing. It does not prove engine behavior.

Good technical documentation should also expose uncertainty. Each chapter ends with room for improvement because the engine is actively modernizing; a print manual should teach current reality without pretending the work is finished.

When the code changes, regenerate or revise the manual from the same source boundaries: startup docs, current source, Agentic references, and rendered QA.

- Keep statements tied to actual code paths and handoff docs.
- Prefer diagrams for ownership and equations for solver math.
- Leave improvement lists as future-work prompts, not vague criticism.

Room for Improvement

The list below is deliberately framed as engineering opportunity rather than criticism. It names the places where the chapter's current architecture can become clearer, smaller, or easier to validate.

	Improvement prompt
[]	Add a repeatable manual regeneration target if the manual becomes a maintained artifact.
[]	Create a small gallery of known-good SkullScope query examples for common physics questions.
[]	Turn validation map tables into links inside the developer docs.
[]	Add a print-friendly replay/diagnostics appendix with concrete command recipes.

Chapter 6: Working With the Engine

Mental model

The best way to change Skullbonez Core is to respect ownership, validate the specific risk, and leave better explanations than you found.

Reading a Subsystem

Start with the owner and the data shape. For runtime work, ask what Run owns and which extracted controller or context narrows the operation. For physics, ask whether the code runs on model mirrors or store records. For rendering, ask whether the pass owns resources or borrows a frame context.

Then ask where the behavior becomes observable. Physics becomes observable through body writeback, CSV baselines, SkullScope traces, debug overlays, and replay. Rendering becomes observable through InfoQueue validation, screenshots, backbuffer capture, and present-time fences.

Finally, find the improvement boundary. If a file contains a transitional facade, understand whether the change narrows ownership or merely renames a bridge.

- Owner first, behavior second, validation third.
- For hot paths, prefer arrays, records, scratch buffers, and side-effect queues.
- For public surfaces, preserve command-line and scene-file compatibility unless a plan says otherwise.

Changing Physics Safely

Physics is deterministic, so even small ordering changes matter. A refactor that preserves final-looking motion can still change byte-exact baselines if it changes broadphase pair order, warm-start cache keys, row iteration order, sleep timing, or final integration order.

The safest physics edits keep hot loops data-oriented, avoid new polymorphic service objects, and commit owner mutations after the pass. If a body needs to wake or a debug record needs to be emitted, queue the fact and apply it at the owning boundary.

When behavior intentionally changes, refresh baselines only from the final Debug executable, committed scene files, and committed config state, then rerun the matching physics gate.

- Never treat copied physics artifacts as trustworthy baselines.
- Do not introduce callback-style bridges into solver or collision loops.
- Use SkullScope queries when diagnosing before changing code.

Changing Rendering Safely

DX12 renderer work is explicit-state work. Look for resource lifetime, descriptor lifetime, upload allocator reuse, fence ordering, command list state, and graph transitions before changing pass behavior.

Visual regressions are measured with DX12 screenshots and zero validation errors. GL/DX11 parity is historical; do not add new runtime dependencies on retired backends to prove a current rendering change.

If a pass needs a resource, decide whether it is a persistent pass-owned resource, a frame-local graph transient, an imported backbuffer/depth target, or a source asset that the backend materializes.

- Graph access vocabulary should describe resource intent before DX12 state translation.
- Descriptors are not resources; they are views into resources.
- Fence completion is the authority for reuse and deferred release.

Writing Helpful Comments and Handoffs

The repository comment standard treats comments as learning aids, not decoration. A good comment names a concept, invariant, lifetime, hazard, unit, validation-sensitive behavior, or local vocabulary that the code cannot express cleanly by itself.

Handoffs should record changed areas, validation decisions, commands run, meaningful results, artifacts updated, and residual risk. Commit notes should explain what changed and why, not merely name files.

The manual follows the same spirit: diagrams teach ownership, equations teach physics/math, and improvement sections name what is still worth doing.

- Comment dense or risky local behavior close to the code.
- Avoid vague migration nouns unless the owner, reason, deletion condition, and checker budget are recorded.
- Keep final reports concise but evidence-backed.

Room for Improvement

The list below is deliberately framed as engineering opportunity rather than criticism. It names the places where the chapter's current architecture can become clearer, smaller, or easier to validate.

	Improvement prompt
[]	Turn this chapter into a shorter onboarding checklist for new contributors.
[]	Add small worked examples for common tasks: add a scene, add an asset, diagnose a contact, add a render pass.
[]	Create a maintained glossary linked from README and Agentic references.
[]	Keep future manual revisions tied to actual source and rendered QA instead of static memory.

Appendix A: Source Landmarks

These paths were used as grounding context for the manual. They are listed so a future reader can reopen the current implementation instead of treating the prose as a substitute for source review.

Area	Primary paths
Startup and rules	AGENTS.md; README.md; Agentic/README.md; Agentic/SessionState.md
Runtime reference	Agentic/Reference/runtime-reference.md
Class structure	Agentic/Reference/skullbonez-core-class-structure.md
Physics overview	Agentic/Reference/physics-overview.md
Runtime frame/render	SkullbonezSource/Runtime/RunFrame.cpp; SkullbonezSource/Runtime/RunRender.cpp
Physics	SkullbonezSource/Physics/SimulationSystem.cpp; PhysicsEngine.*; PhysicsScene.cpp; PhysicsWorld.cpp; PersistentContactSolver.*; SpatialGrid.*
Rendering	SkullbonezSource/Runtime/Render/*; SkullbonezSource/Rendering/RenderGraph.*; SkullbonezSource/Rendering/DX12/*
Scene and assets	SkullbonezSource/Scene/*; SkullbonezSource/Assets/AssetSystem.*; SkullbonezData/scenes/*; SkullbonezData/assets/*

Appendix B: Glossary

Broadphase: Cheap collision phase that finds pairs worth exact testing.

Middle phase: Pair pruning between grid candidates and exact narrowphase.

Narrowphase: Precise collision/contact generation for a candidate pair.

Manifold: Set of contact points, normals, tangents, and metadata for a touching pair.

Persistent row: A solver row with stable feature id and cached impulses across frames.

Warm start: Applying last frame's converged impulse before this frame's solver iterations.

Render graph: API-neutral declaration of render resources, pass uses, transitions, and transient lifetimes.

Descriptor: DX12 binding record that tells shaders how to interpret a resource.

Fence: GPU timeline value that proves when frame resources can be reused or released.

SkullScope: Queryable diagnostics workflow for compact trace inspection.

Appendix C: Equation Index

Fixed timestep accumulator

```
acc = acc + frame_dt * time_scale
while acc >= h: Step(h); acc = acc - h
Skullbonez physics tick: h = 1 / 120 seconds
```

Fixed timestep accumulator.

Velocity-first integration

```
v_next = v + h * M_inverse * F
x_next = x + h * v_next
orientation advances from angular velocity after solver velocity writeback
```

Velocity-first integration.

Contact point velocity

```
vA_p = vA + omegaA cross rA
vB_p = vB + omegaB cross rB
v_rel = vB_p - vA_p, v_n = dot(v_rel, n)
```

Contact point velocity.

Normal row impulse update

```
m_eff = 1 / (invMassA + invMassB + angular terms)
delta_lambda_n = m_eff * (bias - v_n)
lambda_n = max(0, lambda_n + delta_lambda_n)
```

Normal row impulse update.

Tangent friction clamp

```
lambda_t = (lambda_t1, lambda_t2)
length(lambda_t) <= friction_limit
```

Skullbonez clamps the tangent pair as one 2D cone budget.

Tangent friction clamp.

World to clip space

```
p_world = M_model * p_local
p_clip = Projection * View * p_world
```

Shadow/reflection passes use alternate View matrices, not alternate objects.

World to clip space.

Render graph transition rule

```
before = tracked_access(resource)
if before != requested_access: emit transition(before -> requested)
tracked_access(resource) = requested_access
```

Render graph transition rule.

Fence-scoped reuse

```
submit(command_list); signal(frame_fence)
reuse allocator/upload/descriptors only after completed_fence >= frame_fence
deferred releases retire on completed fences.
```

Fence-scoped reuse.

Appendix D: Worked Reading Pages

These pages are short field guides for common reading tasks. They are intentionally procedural so a printed copy can sit beside the code while investigating.

Trace One Fixed Physics Tick

Start at `SimulationSystem::Tick` and identify whether the scene is fixed-step or accumulator-driven. Follow the committed tick into `GameModelCollection::RunPhysics`, then through `PhysicsEngine::Step` and `PhysicsScene::RunPhysics`.

In `PhysicsScene`, watch the model-owner boundary: body records reload from `GameModel` state, collider snapshots refresh only when topology changed, `PhysicsWorld` mutates store data, then body records write back once for downstream consumers.

In `PhysicsWorld`, read the phase order as an invariant: per-frame buffers, force pass, broadphase, object CCD front-end, terrain detection, persistent contact solve, point joints, sleep support, remaining integration, sleep island decision.

- Question to answer: which phase first changes velocity?
- Question to answer: which phase first changes position?
- Question to answer: which phase writes solved store state back to models?
- Validation thought: any phase reorder can change byte-exact physics baselines.

Inspect a Contact Row

A contact row is a compact constraint record. It is not a force object and it is not a callback into the model. Read bodyA/bodyB, featureId, normal, tangent axes, rA/rB arms, penetration, support flags, and accumulated impulses.

Then read precompute. Effective mass translates a desired change in contact velocity into an impulse. Bias represents restitution or Baumgarte separation. The friction limit decides how much tangent impulse can be retained.

Finally read the iteration loop. The row computes relative velocity, updates accumulated normal lambda, clamps it to zero or above, updates tangent lambdas, clamps their vector length, and applies only the delta impulse to scratch velocities.

- Normal row: push apart only.
- Tangent rows: resist sliding within a friction budget.
- Warm start: previous lambda is applied before iteration.
- Cache store: stable feature ids make the next tick start near convergence.

Follow a Render Pass

Start from `Run::Render`. Confirm replay preview state is applied only for the draw and restored afterward. Then look at the `RuntimeRenderInputs` construction to see which backend facets and runtime systems are borrowed for this frame.

Inside `RuntimeRenderer`, find whether the pass is callback-owned through `RenderGraph` or still executes through a direct pass method. The difference affects where resources are declared, where barriers are emitted, and which diagnostics represent the pass.

For a DX12 issue, keep two maps in your head: the logical render graph resource state and the concrete `ID3D12Resource` state. The graph should explain intent; `RenderBackendDX12` should translate that intent into exactly the barriers DX12 needs.

- Question to answer: does the pass own persistent resources, graph transients, or imported targets?
- Question to answer: are descriptors persistent source rows or frame-visible copies?
- Validation thought: renderer changes need zero DX12 validation errors plus screenshot evidence.

Add a Reusable Scene Asset

Reusable placeables belong in asset library data, not in one-off editor-only hardcoding. Start with SkullbonezData/assets and decide whether the asset is a convex hull, compound stack, prop, building, or terrain dressing.

Register the asset library from Run setup with an assetlib.* logical name, then reference it from scene assetInstances. If the asset includes hull data, bake hull metadata so source vertices, faces, edges, mass, and inertia stay current.

When testing, separate scene load evidence from physics/render evidence. A new asset can be a parser/scene change, a physics hull change, a render material change, or all three.

- Prefer assetInstances[] for reusable props.
- Bake changed hulls with tools\bake_hulls.py --write.
- Scene or asset data changes are not documentation-only; choose the matching validation gate before PR prep.

Read a Scene File

Treat scene JSON as declarative intent. First identify playback and simulation fields: fixedStep, frame count, exit behavior, seed, time scale, and world overrides. Then read objects, asset libraries, asset instances, cameras, terrain, debug, cinematic, and UI fields.

Next ask which setup path consumes the data. Authored scene setup preserves model insertion order and validation gates. Generated setup preserves deterministic demo and solver object algorithms.

Finally ask whether a field is physics-affecting or render-only. Cinematic relief and styles should not alter physics terrain. Object materials alter presentation and material response only when the parsed field explicitly does so.

- Bare scene names resolve through SkullbonezData\scenes.
- Style includes should not rebuild models or reset physics.
- Snapshot output is a serialization path, not the source of truth for hand-authored scenes.

Choose a Validation Gate

Validation starts from risk, not habit. If the change is prose-only, repository validation is not required. If it changes physics behavior, run the physics gate. If it changes DX12 rendering, run the DX12 renderer gate. If it touches a hot path, add perf evidence.

Use broad validation only when the scope is broad or uncertain. During iteration, targeted builds or focused probes are acceptable when they answer a specific implementation question, but formal scripts are PR gates.

Always preserve the command and meaningful result lines in the handoff. A future reader should know what was proven, what was not run, and why.

- Docs/manual: document render QA, no repo validation.
- Physics: tools\validate_physics.bat.
- Renderer/shaders: tools\validate_dx12_renderer.bat.
- Unsure at PR gate: tools\agent_validate.bat.

Debug a Physics Regression

Start by deciding whether the regression is deterministic behavior, visual debug output, sleep policy, broadphase reachability, terrain support, or replay restore. That choice determines which artifact is useful.

Prefer SkullScope query output over raw logs. Generate a deterministic trace with `--physics-diag`, run `tools\physics_query.bat` summary and events, then ask focused frame/body/contact/island questions.

If a baseline changed intentionally, regenerate it only from the final Debug executable and committed scene/config state, then rerun the matching gate so the committed baseline is byte-exact against the committed behavior.

- Do not paste or ingest whole CSV/NDJSON/SQLite artifacts unless raw logs are explicitly requested.
- Report trace command, query commands, output sizes, and whether output was truncated.
- Treat a single byte of CSV drift as real behavior until proven intentional.

Update This Manual Safely

Regenerate diagrams and equations from the builder when layout or content changes. The DOCX is the editable master, but the print PDF should be regenerated from the same content so the two artifacts do not drift.

Before changing technical claims, re-open the current source or Agentic reference path named in Appendix A. The manual should be a guide to the implementation, not a fossilized memory of a previous branch.

After generation, render or inspect the final PDF pages. If DOCX-to-PDF conversion is available on the machine, compare the native PDF and the DOCX-derived PDF for pagination differences before sending to print.

- Keep chapter improvement lists actionable.
- Add new equations only when they clarify implementation behavior.
- Update source landmarks when subsystem ownership moves.